Assignment 4: Sequence Modeling and Parsing

Academic Honesty: Please see the course syllabus for information about collaboration in this course. While you may discuss the assignment with other students, **your submission must be your own!**

Goals You will gain experience with basic structured prediction, including the Viterbi algorithm. You'll also look at parse trees, analyze syntactic phenomena, and reason about the behavior of PCFGs.

Deliverables and Submission Please type your solutions and do not submit handwritten work. You will submit your writeup to Gradescope as a PDF or text file of your answers to the questions.

Part 1: HMMs and Tagging (40 points)

Q1 (40 points) Consider the following corpus:

their/N raises/N
he/N raises/V
my/N purses/N

a) List the maximum-likelihood initial state probabilities for an HMM estimated from this data. (Hint: this should be a probability distribution over tags.)

Solution: Use the equation on slide 12 in the "15-Viterbi-NLP-S24" presentation. $\pi(N)$=3/3=1, $\pi(V)$=0/3=0. Note that the initial probability only depends on the first state of each sentence, so $\pi(N)$=5/6 and $\pi(V)$=1/6 is not correct.

b) List the maximum-likelihood transition probabilities for an HMM estimated from this data (including transitions to the STOP symbol).

Solution: Use the equation on slide 12 in the "15-Viterbi-NLP-S24" presentation.

	N	V	STOP
N:	2/5	1/5	2/5
V:	0	0	1

c) Give an example of a grammatical English sentence **and its tags** using the words above that would be assigned zero probability under this HMM (assuming no smoothing is applied).

Solution: Following equation on slide 8 in "15-Viterbi-NLP-S24", one can find that these: he/N purses/V, he/N raises/V purses/N, he/N raises/V their/N purses/N, and other solutions that involve a V to N transition, would be assigned zero probability. Note that this sentence should use the words above, not just random other English words.

For the following question parts, assume that the log probabilities (log base 2) are as follows (these are rounded so they don't quite normalize as you would expect):

Initial	
N	-1
V	-1

Transitions			
	N	V	STOP
y_{i-1} = N	-1	-1.5	-2
y_{i-1} = V	-2	-2.5	-0.5

Emissions					
	their	he	my	raises	purses
N	-4	-2	-3	-3	-2
V	-5	-7	-5	-2	-4

Consider the sentence *he raises purses*

d) Apply the Viterbi algorithm on this data and show your work.

Solution:
Using the Viterbi algorithm outlined on slide 33 in "15-Viterbi-NLP-S24" and slightly modifying it to work with log-probabilites, the scores for N and V are:

N: -3, -7, -10
V: -8, -6.5, -12.5

Note that maximizing log probabilities and maximizing probabilities gives the same sequence as tags. We have talked about why this is the case throughout the semester starting with the introduction of minimizing the negative log-likelihood. We get these values as follows:

$$\text{score}_1(N) = \mathbb{P}(N)\mathbb{P}(\textit{he} \mid N) \Rightarrow \log(\text{score}_1(N)) = \log\mathbb{P}(N) + \log\mathbb{P}(\textit{he} \mid N) = -1 - 2 = -3$$
$$\text{score}_1(V) = \mathbb{P}(V)\mathbb{P}(\textit{he} \mid V) \Rightarrow \log(\text{score}_1(V)) = \log\mathbb{P}(V) + \log\mathbb{P}(\textit{he} \mid V) = -1 - 7 = -8$$

$$\begin{aligned}
\text{score}_2(N) &= \max\Big(\mathbb{P}(N|N)\mathbb{P}(\textit{raises} \mid N) \cdot \text{score}_1(N), \\
&\qquad \mathbb{P}(N \mid V)\mathbb{P}(\textit{raises} \mid N) \cdot \text{score}_1(V)\Big) \\
\Rightarrow \log(\text{score}_2(N)) &= \max\Big(\log\mathbb{P}(N|N) + \log\mathbb{P}(\textit{raises} \mid N) + \log(\text{score}_1(N)), \\
&\qquad \log\mathbb{P}(N \mid V) + \log\mathbb{P}(\textit{raises} \mid N) + \log(\text{score}_1(V))\Big) = \\
&= \max(-1-3-3, -2-3-8) = -7
\end{aligned}$$

Similarly, we would get $\text{score}_2(V) = -6.5$, $\text{score}_3(N) = -10$, and $\text{score}_3(V) = -12.5$. Finally, $\log(\text{score}_3(N)) = -10$ is the maximum score among the tags (N, V) at the final step; therefore, the tag for purses is N. Calculating $\log(\text{score}_3(N)) = -10$ would reveal that the score is derived from $\log(\text{score}_2(N))$. Consequently, the tag for raises is also N. Following this pattern, $\log(\text{score}_2(N))$ depends on $\log(\text{score}_1(N))$, leading us to tag he as N. All together, the HMM and Viterbi predict NNN for the given sentence.

e) What is the highest posterior probability tag sequence for this sentence (now including the STOP transition)? **Is this sequence consistent with your interpretation of the sentence above?**

Solution: NNN, which is not consistent with our interpretation (NVN).

Part 2: Syntactic Parsers (60 points)

Q2 (20 points) In this part, you'll look at how parsers work in practice. You'll be using the demo for the Stanford CoreNLP constituency and dependency parsers—https://corenlp.run/—which are strong parsing models available as web demos. Under "Annotations" you should choose "part-of-speech" and either "constituency parse" and/or "dependency parse".

Dependency parsers produce relations between *head* words (parents) and *modifier* words (children). There are no intermediate categories like in constituency parsing; the sentence itself and its dependency arcs becomes a directed tree. Verbs are the heads of sentences. Subjects and objects of verbs are then modifiers of those verbs. Adjectives and nouns modify "head" nouns (e.g., for *art museum*, *museum* is the head and *art* is a modifier). These relationships are depicted as arrows going from parents to children.

a) For the sentences *I ate spaghetti with chopsticks* and *I ate spaghetti with meatballs*, describe the following. (1) Which of the two possible interpretations does the **constituency parser** choose for each sentence? Is it correct? (2) Do the same analysis for the **dependency parser**. (You should be able to understand the behavior by consulting the explanations of dependencies above.) (3) Are you surprised by the models' behavior here? Comment on what you see.

Solution: The dependency parser always attaches *with X* to *ate*. The constituency parser similarly attaches the PP to the VP. Both parsers are consistent regardless of what the last word is in these cases, and therefore do not parse the *with meatballs* case correctly, which is a bit surprising for high-performing models.

b) Find a new example that the **constituency parser** parses incorrectly. Include the example, its parse, and describe what is incorrect about the parse. Hint: you can think of what makes sentences ambiguous or try to find complicated sentences.

Solution: Many possible solutions here. Typical garden path sentences like *the horse raced past the barn fell* work; this should be interpreted as *[the horse (that) was raced past the barn] fell*. Other sentences with unusual word usages in the same vein as the *teacher strikes idle kids* example are good too.

Q3 (20 points) Consider the following PCFG (bracketed numbers are probabilities):

NP $\rightarrow$ NP CC NP [0.3]
NP $\rightarrow$ NP PP [0.3]
NP $\rightarrow$ NNS [0.4]
PP $\rightarrow$ P NP [1.0]

NNS $\rightarrow$ cats [1.0]
CC $\rightarrow$ and [1.0]
P $\rightarrow$ in [1.0]

Define a PCFG with these rules, the nonterminals $\{$NP, PP, NNS, CC, P$\}$, terminals $\{$*cats, and, in*$\}$, and root symbol NP. For all question parts, provide justification so we can give partial credit as appropriate.

a) For the sentence *cats and cats*, how many valid syntactic parses (nonzero probability under this grammar) are there?

Solution: You could work out the CKY algorithm ("16-ConstituencyParsing-NLP-S24") or see manually since the grammar is small, to find out there is just one: (NP (NP (NNS cats)) (CC and) (NP (NNS cats)))

b) For the sentence *cats and cats in cats*, how many valid syntactic parses (nonzero probability under this grammar) are there?

Solution: You could work out the CKY algorithm ("16-ConstituencyParsing-NLP-S24"), or see manually since the grammar is small, there are two parses for this one:

```
(NP
 (NP (NP (NNS cats)) (CC and) (NP (NNS cats)))
 (PP (P in) (NP (NNS cats)))
)
```

```
(NP
 (NP (NNS cats))
 (CC and)
 (NP (NP (NNS cats)) (PP (P in) (NP (NNS cats))))
)
```

Q4 (20 points) Consider the sentence *The cat chased the mouse through the garden*. Your task is to construct its dependency structure using the transition-based dependency parsing algorithm. You will make the classification decisions manually.

As you create links between words, you'll decide on the type of relationship (or dependency) they share. Please refer to the UD guidelines for possible relation types: https://universaldependencies.org/u/dep/index.html. Don't worry if you're unsure about the exact labels for each relation. Make your best guess based on the information available.

Show your work step by step.

Solution: Follow "17-DependencyParsing-NLP-S24" slides (Prof Vivek Srikumar's slides linked in them).